

Chapter 13: Properties

By Victoria Gonda

In Chapter 11, you were introduced to **properties** as the data members of Kotlin classes and objects. They are often used to describe the attributes of the object or hold state.

Open the starter project for this chapter to continue learning.

In the example below, the Car class has two properties, both constants that store String values:

```
class Car(val make: String, val color: String)
```

The two properties of Car are supplied in the primary constructor, and they store different string values for each instance of Car.

Properties can also be set up with custom **accessors**, also known as **getters** and **setters**. The properties supplied in the class primary constructor use default implementations of the accessors, storing the data in a **backing field**.

In this chapter, you'll learn much more about properties. You'll learn about property initializers, custom accessors, delegated properties, late initialization of properties and extension properties.

Constructor properties

As you may have guessed from the example in the introduction, you're already familiar with many of the features of properties. To review, imagine you're building an address book. The common unit you'll need is a `Contact`.

Add this class to your Kotlin file:

```
class Contact(var fullName: String, var emailAddress: String)
```

You can use this class over and over again to build an array of contacts, each with a different value. The properties you want to store are an individual's full name and email address.



These are the properties of the `Contact` class. You provide a data type for each, but opt not to assign a default value, because you plan to assign the name and email address upon initialization. After all, the values will be different for each instance of `Contact`.

Create a contact in `main()`:

```
val contact = Contact(  
    fullName = "Grace Murray",  
    emailAddress = "grace@navy.mil"  
)
```

You create an object by passing values as arguments into the class primary constructor.

As with any function call not using default values, using named arguments in the primary constructor is optional. Because named arguments are optional, you could also create the class like this:

```
val contact = Contact("Grace Murray", "grace@navy.mil")
```

Now, add the following statements:

```
val name = contact.fullName // Grace Murray
val email = contact.emailAddress // grace@navy.mil
```

Here, you're accessing the properties. You can access the individual properties using dot notation.

Now, assign a new value to the full name. When Grace married, she changed her last name:

```
contact.fullName = "Grace Hopper"
val grace = contact.fullName // Grace Hopper
```

Combining the dot notation with = assignment, you can assign new values. You can assign values to properties as long as they're defined as variables.

If you'd like to prevent a value from changing, you can define a property as a constant instead using `val`.

Consider this contact class:

```
class Contact2(var fullName: String, val emailAddress: String)
```

Notice that the `emailAddress` property uses `val` instead of `var`.

Now, look what happens when you try to change the value:

```
var contact2 = Contact2(
    fullName = "Grace Murray",
    emailAddress = "grace@navy.mil"
)

// Error: Val cannot be reassigned
contact2.emailAddress = "grace@gmail.com"
```

Once you've initialized an instance of the `Contact2` class, you can't change `emailAddress`.

Default values

If you can make a reasonable assumption about what the value of a property should be when the type is initialized, you can give that property a default value.

It doesn't make sense to create a default name or email address for a contact, but imagine there's a new property type to indicate what kind of contact it is

Create a contact class that has a default value:

```
class Contact3(  
    var fullName: String,  
    val emailAddress: String,  
    var type: String = "Friend"  
)
```

By assigning a value in the definition of type, you give this property a default value. Any contact created will automatically be a friend, unless you change the value of type to something like “Work” or “Family”:

```
var contact3 = Contact3(  
    fullName = "Grace Murray",  
    emailAddress = "grace@navy.mil"  
)
```

This contact3 has the type “Friend”. You could change that by reassigning the type:

```
contact3.type = "Work"
```

Or supplying the type when you initially create it:

```
var workContact = Contact3(  
    fullName = "Grace Murray",  
    emailAddress = "grace@navy.mil",  
    type = "Work"  
)
```

Property initializers

Properties can also be initialized outside of the primary constructor, using literals and values passed into the primary constructor, using a **property initializer**.

Consider the Person class:

```
class Person(val firstName: String, val lastName: String) {  
    val fullName = "$firstName $lastName"  
}
```

In Person, the fullName property is initialized using the values that are passed into the primary constructor:

```
val person = Person("Grace", "Hopper")  
person.fullName // Grace Hopper
```

You can set the value of properties with their declaration, as in `Person`, and also in the `init` block like below:

```
class Address {  
    var address1: String  
    var address2: String? = null  
    var city = ""  
    var state: String  
  
    init {  
        address1 = ""  
        state = ""  
    }  
}
```

In `Address`, the `address2` and `city` properties are initialized in their declaration. The `address1` and `state` properties are initialized inside of `init`. Since all four properties of `Address` are given values inside the class definition, you can create an `Address` instance using an empty constructor call:

```
val address = Address()
```

Custom accessors

Many properties work just fine with the default accessor implementation, in which dot notation returns the value directly and an assignment statement just sets the value. Properties can also be defined with custom getter and setter methods. If a custom setter is provided, then the property must be declared as a `var`.

Custom getter

The measurement for a TV is the perfect use case for a custom accessor. The industry definition of the screen size of a TV isn't the screen's height or width, but its diagonal measurement.

Add this class:

```
class TV(var height: Double, var width: Double) {  
    // 1  
    val diagonal: Int  
    get() {  
        // 2  
        val result = Math.sqrt(height * height + width * width)  
        // 3  
    }  
}
```

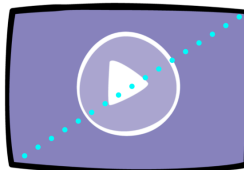
```
        return result.roundToInt()  
    }  
}
```

Going through this code one step at a time:

1. You use an `Int` type for your `diagonal` property. Although height and width are each a `Double`, TV sizes are usually advertised as nice, round numbers such as 50" rather than 49.52". Instead of the usual assignment operator `=` to assign a value as you would for a normal property, you use the `get()` function and curly braces to enclose your property's calculation.
2. Once you have the width and height, you can use the Pythagorean theorem to calculate the length of the diagonal. You use the `Math.sqrt()` method to calculate the diagonal.
3. You return the result as a rounded `Int` using `roundToInt()`: if the decimal is 0.5 or above, it rounds up; otherwise it rounds down. Had you converted `result` directly to `Int` without rounding first, the result would have been truncated, so 109.99 would have become 109.

Note: You need to add the import `import kotlin.math.roundToInt` to the top of your file to use `roundToInt()`.

Since you've provided a custom getter, no value is stored for `diagonal`; it is simply returned based on a calculation. From outside of the class, a property with a custom getter can be accessed just like any other property.



Test this with the TV size calculation. Add this code to `main()`:

```
val tv = TV(height = 53.93, width = 95.87)  
val size = tv.diagonal // 110
```

You have a 110-inch TV.

Let's say you decide you don't like the standard movie aspect ratio and would instead prefer a square screen.

Add this modification to your TV:

```
tv.width = tv.height  
val diagonal = tv.diagonal // 76
```

You cut off some of the screen width to make it equivalent to the height. Now you *only* have a 76-inch square screen. The computed property automatically provides the new value based on the new width.

Mini-exercise

Do you have a television or a computer monitor? Measure the height and width, plug it into a TV object, and see if the diagonal measurement matches what you think it is.

Custom setter

The property you wrote in the previous section is called a **read-only property**. It has a block of code to compute the value of the property: the custom getter. It's also possible to create a **read-write property** with two blocks of code: a custom getter and a **custom setter**. This setter works differently than you might expect. As the property has no place to store a value, the setter usually sets one or more related *other* properties indirectly.

Update your diagonal property to look like this:

```
// 1  
var diagonal: Int  
// 2  
get() {  
    val result = Math.sqrt(height * height + width * width)  
    return result.roundToInt()  
}  
set(value) {  
    // 3  
    val ratioWidth = 16.0  
    val ratioHeight = 9.0  
    // 4  
    val ratioDiagonal = Math.sqrt(  
        ratioWidth * ratioWidth + ratioHeight * ratioHeight  
    )  
    height = value.toDouble() * ratioHeight / ratioDiagonal  
    width = height * ratioWidth / ratioHeight  
}
```

Here's what's happening in this code:

1. You've changed `diagonal` to be a `var` instead of a `val`, since you're giving it a setter to change the value.
2. You use the same code as before to compute the value in the getter.
3. For a setter, you usually have to make some kind of assumption. In this case, you provide a reasonable default value for the screen ratio, in this case 16×9 .
4. The formulas to calculate a height and width, given a diagonal and a ratio, are a bit deep. You could work them out with a bit of time, but we've done the dirty work for you and provided them here.

The important parts to focus on in the formulas for height and width are:

- The `value` parameter to the custom setter lets you use whatever value was passed in during the assignment.
- Since the `value` is an `Int`, you first convert it to a `Double` using `toDouble()`.
- Once you've done the calculations, you assign the height and width properties of the `TV` object.

Now, in addition to setting the height and width directly, you can set them *indirectly* by setting the `diagonal` property. When you set this value, your setter will calculate and store the height and width.

Notice that there's no `return` statement in a setter — it only modifies the other stored properties. With the setter in place, you have a nice little screen size calculator. Try it out using the following code:

```
tv.diagonal = 70
println(tv.height) // 34.32...
println(tv.width)  // 61.01...
```

Now you can discover the biggest TV that will fit in your cabinet or on your shelf.

Companion object properties

In the previous section, you learned how to associate properties with instances of a particular class. The properties on your instance of `TV` are separate from the properties on someone else's instance of `TV`.

However, the class *itself* may also need properties that are common across all instances. As you saw in the previous chapter, these properties are put into the **companion object** for the class. Companion object properties are similar to but not exactly like **static** properties that you find in other languages.

Imagine you're building a game with many levels. Each level has a few attributes, passed in the primary constructor

Create this scenario in your code:

```
class Level(  
    val id: Int,  
    var boss: String,  
    var unlocked: Boolean  
) {  
    companion object {  
        var highestLevel = 1  
    }  
}  
  
val level1 = Level(id = 1, boss = "Chameleon", unlocked = true)  
val level2 = Level(id = 2, boss = "Squid", unlocked = false)  
val level3 = Level(id = 3, boss = "Chupacabra", unlocked =  
false)  
val level4 = Level(id = 4, boss = "Yeti", unlocked = false)
```

You can use a companion object property to store the game's progress as the player unlocks each level. Here, `highestLevel` is a property on `Level` itself rather than on the instances.

That means you don't access this property on an instance:

```
// Error: Unresolved reference  
// Can't access members of the companion object on an instance  
val highestLevel = level3.highestLevel
```

Instead, you access it on the class itself. Add this to your code:

```
val highestLevel = Level.highestLevel // 1
```

Using a companion object property means you can retrieve the same property value from anywhere in the code for your app or algorithm. The game's progress is accessible from any level or any other place in the game, like the main menu.

For Kotlin on the JVM, you can use the `@JvmStatic` annotation to force a property to be a static field in the bytecode, with static getters and setters. This will allow you to avoid having to use the singleton name in your Java code.

Update `Level` to look like this alternative version:

```
class Level(  
    val id: Int,  
    var boss: String,  
    var unlocked: Boolean  
) {  
    companion object {  
        @JvmStatic var highestLevel = 1  
    }  
}
```

Now, from Java, you can access `highestLevel` as follows:

```
Level.getHighestLevel() // Fine, thanks to @JvmStatic  
Level.Companion.getHighestLevel() // Fine too, and necessary if  
@JvmStatic were not used
```

`@JvmStatic` gives you nicer code in Java, and doesn't change how you access `highestLevel` in Kotlin.

Delegated properties

Most of the property initialization you've seen so far has been straightforward. You provide an initializer for a property, for example, as a literal value or default value, or you use custom accessors to compute the value.

For more complicated initializations, you may want to pass the initialization off to another object, or delay the initialization from when the instance is created. You may also want to observe when a property changes. For these cases, you can use **delegated properties**, which are indicated with the use of the `by` keyword.

Observable properties

For your `Level` implementation, it would be useful to automatically set the `highestLevel` when the player unlocks a new one. For that, you'll need a way to listen to property changes. Thankfully, you can use a delegated property **observable** to provide a callback for when the property changes.

Create a new class to practice this:

```
class DelegatedLevel(val id: Int, var boss: String) {  
    companion object {  
        var highestLevel = 1  
    }  
}
```

```
    }  
    var unlocked: Boolean by Delegates.observable(false) {  
        _, old, new ->  
        if (new && id > highestLevel) {  
            highestLevel = id  
        }  
        println("$old -> $new")  
    }  
}
```

You specify the property observer using `by Delegates.observable()`, whose first parameter is the initial value of the property.

Note: Add the import `import kotlin.properties.Delegates` to use `Delegates`.

In this case, `unlocked` is initially `false`. The second parameter to `observable()` is a lambda with three arguments, the first of which is the property object itself (which you ignore), and the second and third of which are the old and new value of the property respectively.

The lambda is invoked after the value of `unlocked` is changed, so `new` indeed has the new value.

Add the following to try out your delegate:

```
val delegatedlevel1 = DelegatedLevel(id = 1, boss = "Chameleon")  
val delegatedlevel2 = DelegatedLevel(id = 2, boss = "Squid")  
  
println(DelegatedLevel.highestLevel) // 1  
  
delegatedlevel2.unlocked = true  
  
println(DelegatedLevel.highestLevel) // 2
```

Now, when the player unlocks a new level, it will update the `highestLevel` of `DelegatedLevel` if the level is a new high.

Limiting a variable

You can also use delegated property observers to limit the value of a variable. Say you had a light bulb that could only support a maximum current flowing through its filament.

Add this class:

```
class LightBulb {  
    companion object {  
        const val maxCurrent = 40  
    }  
    var current by Delegates.vetoable(0) {  
        _, new ->  
        if (new > maxCurrent) {  
            println(  
                "Current too high, falling back to previous setting."  
            )  
            false  
        } else {  
            true  
        }  
    }  
}
```

In this example, you're using `by Delegates.vetoable()` and passing an initial value. The lambda callback passed to `vetoable()` returns a Boolean indicating whether the value should be allowed to be changed. If the current flowing into the bulb exceeds the maximum value, it will revert to its last successful value.

Give it a try:

```
val light = LightBulb()  
light.current = 50  
var current = light.current // 0  
light.current = 40  
current = light.current // 40
```

You try to set the light bulb to 50 amps, but the bulb rejected that input. Pretty cool!

Note: Do not confuse delegated property observers with getters and setters. Delegated properties cannot have custom accessors. These are completely different concepts!

Lazy properties

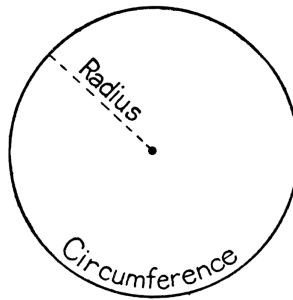
If you have a property that might take some time to calculate and you don't want to slow things down until you actually need the property, say hello to **lazy properties**.

These could be useful for such things as downloading a user's profile picture or making a serious calculation.

Add this example of a `Circle` class that uses `pi` in its circumference calculation:

```
class Circle(var radius: Double = 0.0) {  
    val pi: Double by lazy {  
        ((4.0 * Math.atan(1.0 / 5.0)) - Math.atan(1.0 / 239.0)) *  
        4.0  
    }  
    val circumference: Double  
        get() = pi * radius * 2  
}
```

Here, you're not trusting the value of `pi` available to you from the standard library; you want to calculate it yourself.



Now, create a new `Circle` instance, and the `pi` calculation won't run yet:

```
val circle = Circle(5.0) // got a circle, pi has not been run
```

The calculation of `pi` waits patiently until you need it. Only when you ask for the `circumference` property is `pi` calculated and assigned a value.

Access the value of the `circumference`, and this calculation will happen:

```
val circumference = circle.circumference // 31.42  
// also, pi now has a value
```

Since you've got eagle eyes, you've noticed that the delegated property `pi` uses a `by lazy { }` pattern to calculate its value. The trailing parentheses are a lambda that initializes the value for `pi`. But since `pi` is marked as `by lazy`, this calculation is postponed until the first time you access the property.

For comparison, `circumference` is a non-delegated property and therefore is calculated every time it's accessed. You expect the `circumference`'s value to change if the `radius` changes. `pi`, as a lazy property, is only calculated the first time. That's great, because who wants to calculate the same thing over and over again?

When you first initialize the `Circle` instance, the `pi` property effectively has no value. Then when some part of your code requests the property, its value will be calculated. The value only changes once, so you can use `val` on the property.

Mini-exercises

Of course, you should absolutely trust the value of `pi` from the standard library. It's a constant in the standard library, and you can access it as `kotlin.math.PI`. Given the `Circle` example above:

1. Remove the lazy property `pi`. Use the value of `pi` from the standard library instead.
2. Add a lazy property to `Circle` to calculate the area of the circle. Remember, the equation for the area of a circle is **`pi*radius*radius`**.

lateinit

If you just want to denote that a property will not have a value when the class instance is created, then you can use the **`lateinit`** keyword.

Create this `Lamp` class that has a `LightBulb` property declared as a `lateinit var`:

```
class Lamp {  
    lateinit var bulb: LightBulb  
}
```

Since the property has no value when the class instance is initialized, and the property will be changed at some later time, you must use `var` with `lateinit` and not `val`.

So, what if you bought a new lamp and then came home to discover that you had no spare bulbs?

```
val lamp = Lamp()  
// ... lamp has no lightbulb, need to buy some!  
  
println(lamp.bulb)  
// Error: kotlin.UninitializedPropertyAccessException:  
// lateinit property bulb has not been initialized  
  
// ... bought some new ones  
lamp.bulb = LightBulb()
```

If you try to access the `lateinit bulb` property before it's been initialized, you'll get an exception. Ouch!

Once you've assigned a value to `bulb`, you can turn the lights on and off as much as you'd like.

Extension properties

A circle has a radius, diameter and circumference that are all related to one another. But the `Circle` class above only includes the radius and circumference. It would be nice if the circle could tell you its diameter too, without you having to perform the calculation every time.

But what if the `Circle` class were provided to you in a library, so you could not modify its definition to add a diameter property? Kotlin uses **extension properties** to allow you to add such functionality without changing the class definition.

To add an extension property, create a new property with the property name appended to the class name, like so:

```
val Circle.diameter: Double
    get() = 2.0 * radius
```

You've created an extension property named `diameter` on the `Circle` class, and are providing a custom getter for `diameter`. Extension properties do not have backing fields, so you can only define them using custom accessors.

You can access the extension property just like any other property defined within the class:

```
val unitCircle = Circle(1.0)
println(unitCircle.diameter) // > 2.0
```

Nice! You no longer have to remember that complicated 2x relationship between radius and diameter yourself.

Challenges

Here are some challenges to test your new knowledge. Take a peek at the challenge solutions in the chapter materials if you need a hint while completing them.

Challenge 1

Rewrite the `IceCream` class below to use default values and lazy initialization:

```
class IceCream {  
    val name: String  
    val ingredients: ArrayList<String>  
}
```

1. Use a default value for the `name` property.
2. Lazily initialize the `ingredients` list.

Challenge 2

At the beginning of the chapter, you saw a `Car` class. Dive into the inner workings of the car and rewrite the `FuelTank` class below with delegated property observer functionality:

```
class FuelTank {  
    var level = 0.0 // decimal percentage between 0 and 1  
}
```

1. Add a `lowFuel` property of `Boolean` type to the class.
2. Flip the `lowFuel` `Boolean` when the `level` drops below 10%.
3. Ensure that when the tank fills back up, the `lowFuel` warning will turn off.
4. Add a `FuelTank` property to `Car` and fill the tank. Then drive around for awhile.

Key points

- **Properties** are variables and constants that are part of a named type.
- **Default values** can be used to assign a value to a property within the class definition.
- **Property initializers** and the **init block** are used to ensure that the properties of an object are initialized when the object is created.
- **Custom accessors** are used to execute custom code when a property is accessed or set.
- The **companion object** holds properties that are universal to all instances of a particular class.
- **Delegated properties** are used when you want to observe, limit or lazily create a property. You'll want to use **lazy properties** when a property's initial value is computationally intensive or when you won't know the initial value of a property until after you've initialized the object.
- **lateinit** can be used to defer setting the value of a property reference until after the instance is created.
- **Extension properties** allow you to add properties to a class outside of the class definition, for example, if you're using a class from a library.

Where to go from here?

You saw the basics of properties while learning about classes, and now you've seen the more advanced features they have to offer. You've already learned a bit about methods in the previous chapters and will learn even more about them in the next one!